

Module 5: Code Optimization & Code Generation

Basic Blocks, CFG, DAG, Loop Optimizations, Data-Flow Equations & Register Coloring

Module V: Code Generation & Optimization – Complete 12-Topic Syllabus Tracker

Topic 1: Addresses of Code and Data in Assembly Code

Topic 3: Construction of Basic Blocks (3 Leader Rules)

Topic 5: Machine-Independent Local Optimizations

Topic 7: Unreachable Code Elimination

Topic 9: Constant Propagation Optimization

Topic 11: Common Subexpression Elimination (CSE)

Topic 2: Correlation of Assembly Code with Source Code

Topic 4: Control Flow Graph (CFG Nodes & Edges)

Topic 6: Machine-Independent Global Optimizations

Topic 8: Constant Folding Optimization

Topic 10: Loop-Invariant Code Motion (Hoisting)

Topic 12: Dead Code Elimination

Topic 1 & 2: Assembly Code Addresses & Source Correlation

The final code generator translates intermediate three-address representations into target machine instructions (assembly). The compiler maintains a strict correspondence between source language abstractions, intermediate variables, and physical hardware addresses:

```
Source Code: x = a + b;  
            ↓  
Intermediate: t1 = a + b; x = t1;  
            ↓  
Assembly:    MOV R1, a ; ADD R1, b ; MOV x, R1
```

Symbolic Address Mapping: Source identifiers are assigned relative stack frame offsets or static data segment addresses.

Source-Level Debugging: Line number tables and DWARF metadata allow debuggers (GDB) to map assembly crash addresses directly to source code lines.

Topic 3 & 4: Construction of Basic Blocks & Control Flow Graphs (CFG)

Algorithm: Partitioning Three-Address Code into Basic Blocks (3 Leader Rules)

Step 1: Identify Leaders (First instruction of a basic block):

1. The first three-address instruction of the intermediate code is a Leader.
2. Any instruction that is the target of a conditional or unconditional jump (`goto L`) is a Leader.
3. Any instruction that immediately follows a conditional or unconditional jump is a Leader.

Step 2: Form Basic Blocks:

For each Leader, its Basic Block consists of the Leader and all subsequent instructions up to (but not including) the next Leader or the end of the program.

Step-by-Step Solved Problem: Basic Block Partitioning Trace

```
(1) a = b + c      ← Leader (Rule 1: First instruction)
(2) if a < d goto 5
(3) x = y + z      ← Leader (Rule 3: Follows conditional jump at line 2)
(4) goto 6
(5) x = 0          ← Leader (Rule 2: Target of goto 5 in line 2)
(6) print x        ← Leader (Rule 2: Target of goto 6 in line 4)
```

Resulting Basic Blocks:

- **Block B_1** : Instructions (1) and (2)
- **Block B_2** : Instructions (3) and (4)
- **Block B_3** : Instruction (5)
- **Block B_4** : Instruction (6)

Topic 5 & 6: Local vs. Global Optimizations

Scope	Operating Domain & Algorithms	Key Advantages & Tradeoffs
Local Optimization	Operates strictly within a single basic block (straight-line code without branches). Analyzed via Directed Acyclic Graphs (DAG).	Fast, linear time complexity; requires zero data-flow analysis equations across blocks.
Global Optimization	Operates across multiple basic blocks throughout the entire Control Flow Graph (CFG). Uses iterative Data-Flow Analysis equations.	Achieves massive performance gains across loops and procedures; higher compile-time cost.

Topic 7 – 12: Principal Machine-Independent Optimizations

Optimization Pass	Transformation Rule	Before → After Example
7. Unreachable Code Elimination	Deletes basic blocks and instructions that have zero incoming edges in the CFG.	<code>`return x; y = 10;`</code> → <code>`return x;`</code>
8. Constant Folding	Evaluates operations on compile-time constants upfront at compile time.	<code>`x = 10 * 20 + 4`</code> → <code>`x = 204`</code>
9. Constant Propagation	Substitutes known constant variable values into subsequent expressions.	<code>`pi = 3.14; r = 2; a = pi * r;`</code> → <code>`a = 3.14 * 2;`</code>
10. Loop-Invariant Code Motion	Hoists computations that evaluate to the exact same value in every loop iteration outside the loop header.	<code>`while (i < n) { x = a * b; i++; }`</code> → <code>`x = a * b; while (i < n) { i++; }`</code>
11. Common Subexpression Elimination (CSE)	Eliminates redundant re-evaluations of expressions whose operand values have not changed.	<code>`t1 = a + b; ...; t2 = a + b;`</code> → <code>`t2 = t1;`</code>
12. Dead Code Elimination	Removes assignments whose computed values are never subsequently read or used anywhere in the program.	<code>`x = 10; x = 20; print(x);`</code> → <code>`x = 20; print(x);`</code>



M5 Active Recall & Exam-Important Question Bank

Q1. Differentiate between Unreachable Code and Dead Code with code examples. (6 Marks)

- **Unreachable Code:** Code that can *never be executed* at runtime because no control flow path can ever reach it (e.g., instructions following an unconditional `return` or `goto` without a label):

`return 5; x = 10;` → `x = 10` is unreachable.

- **Dead Code:** Code that *is executed*, but its computation is useless because the computed value is never referenced by any subsequent instruction:

`x = a + b; x = 20; print(x);` → `x = a + b` executes, but is completely dead because `x` is immediately overwritten before being read.